

Learned Embedding Cost cho Cell-Ancestry Inference

Tổng hợp nghiên cứu, insight và research question — hướng đến CVPR Bio-Image track

2026-09-29

Tóm tắt

Một learned metric-embedding cost — thay cho cost thủ công (khoảng cách pixel, OT thủ công) mà mọi baseline trước đây dùng — kết hợp với decode bằng balanced entropic optimal transport (OT), lần đầu tiên thắng hoặc hòa mọi baseline (LAP-capacitated, PyUAT ILP) ở **cả ba** chế độ thừa dữ liệu (ends, s16, s32) cùng lúc, thay vì phải đánh đổi giữa chúng như trước. Kết quả chốt: `embed_v3_ot_e0.5_tinf` đạt ends=0.400, s16=0.744, s32=0.668, so với PyUAT (0.313/0.151, 0.802, 0.589) và LAP2 cũ (0.373, 0.706, 0.577). Vẫn còn thua PyUAT ở s16 (~6 điểm) và giới hạn lớn nhất hiện nay là chỉ chạy trên 1 dataset (OIAM) với oracle mask — hai điểm này quan trọng hơn khoảng cách s16 khi đánh giá mức độ sẵn sàng cho CVPR Bio-Image track.

1. Bài toán & Động lực

Benchmark: ancestor-chaining trên time-lapse imaging thừa mẫu — với mỗi cell tại khung hình quan sát cuối b , dự đoán ancestor của nó tại khung hình neo a , đánh giá qua `score_window` (macro-average theo window trong movie, rồi theo movie), trên 3 mức độ thừa: ends (chỉ 2 khung hình, cách xa nhau), s16, s32 (khoảng cách trung bình).

Baseline có sẵn trước khi bắt đầu hướng này:

- Chaining bằng khoảng cách pixel thô + LAP (Hungarian có capacity) hoặc entropic OT thủ công (`cellanc/transport.py`) — cost hand-set, không học được đặc trưng hình dạng/chuyển động.
- **PyUAT** (`experiments/pyuat_run.py`): ILP tracker khai thác shape + motion/division likelihood đồng thời trên nhiều khung hình. Thắng áp đảo baseline LAP/OT ở s16/s32 (~0.6–0.8 so với ~0.45–0.55), nhưng **collapse tại ends** (2 khung hình) vì thiếu quan sát để fit likelihood — rơi xuống dưới cả nearest-neighbour.

Khoảng trống: không có phương pháp nào thắng ở cả hai đầu (ends lẫn moderate sparsity) cùng lúc. Ý tưởng cốt lõi của hướng nghiên cứu này: thay cost thủ công bằng một **learned embedding cost** duy nhất, huấn luyện trên curriculum nhiều mức Δt (1 đến 132 phút) để không cần chuyển đổi phương pháp theo độ thừa dữ liệu.

2. Phương pháp

Kiến trúc `Embed` (`cellanc/embed.py`):

```
phi(features) -> R^16          embedding mỗi cell (MLP 2 lớp ẩn)
g(phi(cell), Δt)              residual motion predictor: embedding dự đoán Δt phút sau
cost(i@t, j@t+Δt) = ||g(phi(i),Δt) - phi(j)||^2 * exp(-log_tau)
```

Huấn luyện (`experiments/train_embed.py`): listwise softmax cross-entropy — mỗi cell target so với mọi ứng viên ancestor trong cùng khung nguồn — trên curriculum GAPS = [1,2,4,8,16,32,64,132] (phút = khung hình trên OIAM). Việc feed Δt tường minh và huấn luyện trên mọi mức gap (không chỉ khung liền

kề) là điểm khắc phục trực tiếp nhược điểm “collapse tại ends” của PyUAT: model buộc phải học được đặc trưng một gia đình tế bào ở mọi khoảng cách, không riêng khung liền kề.

Feature set ($N_FEATURES$, tiến hóa qua các version, xem mục 3): vị trí chuẩn hóa, trục hình dạng + log kích thước, mật độ cục bộ ($k=5$ trung bình), vận tốc (từ khung quan sát liền trước cùng track id, có dropout khi huấn luyện để robust với cell mới xuất hiện), khoảng cách hàng xóm gần nhất (cờ báo chia tách tế bào).

Chỉ dùng oracle feature: hình dạng lấy từ oracle mask qua `gt_mapping` — đây là phát triển phương pháp trên nền vị trí oracle, chưa phải pipeline detector thật (xem mục 7, Hạn chế).

3. Thí nghiệm: Feature Ablation

Version	$N_FEATURES$	Thêm gì	Insight
v0	7	vị trí, trục, log shape, log spacing	Go/no-go ban đầu cho learned cost
v1	7	(giống v0, huấn luyện lâu hơn, <code>t-stride=3</code>)	Cải thiện nhỏ — huấn luyện lâu hơn không phải nút thắt chính
v2	10	+ vận tốc (<code>vy</code> , <code>vx</code> , <code>has_velocity</code>)	Cải thiện lớn nhất session: s16 từ 0.595 → 0.707
v3	11	+ <code>log_nearest_dist</code> (cờ chia tách)	Cải thiện biên: s32 chỉ +0.014

Vì sao vận tốc là đòn bẩy lớn nhất: một chẩn đoán trực tiếp (so sánh độ chính xác single-hop) cho thấy embedding không có vận tốc thiếu đúng loại tín hiệu mà motion-likelihood của PyUAT có — v1 chỉ thắng PyUAT ở single-hop khi PyUAT phải suy luận qua chuỗi 8-hop. Thêm vận tốc cho phép model học “quán tính chuyển động” của mỗi tế bào mà không cần fit ILP nhiều khung hình.

Giả thuyết “compounding error” đã bị bác bỏ: ban đầu nghĩ điểm yếu s16/s32 đến từ lỗi tích lũy qua nhiều bước chaining tham lam ($0.708^8 \approx 0.057$ dự đoán quá bi quan so với ~ 0.58 – 0.60 thực tế đo được). Nguyên nhân thật là chất lượng cost per-hop yếu ở Δt trung bình, không phải compounding — dẫn trực tiếp đến việc thêm vận tốc.

4. Thí nghiệm: Decode Strategy

Giữ nguyên checkpoint đã huấn luyện, chỉ đổi cách decode cost matrix thành lineage:

Decode	Cơ chế	Kết quả
nearest lap (<code>cap=2/4/soft</code>)	argmin mỗi hop, không capacity Hungarian có capacity mỗi hop (row-replication)	Baseline Chứng minh được là tối ưu toàn cục cho mục tiêu tổng chi phí theo chuỗi, với ràng buộc capacity chỉ theo từng hop
soft	Belief propagation qua column-softmax	Luôn kém hơn lap2 ở mọi nhiệt độ (thiếu ràng buộc capacity cứng, temperature càng cao càng làm mờ kết quả)

Decode	Cơ chế	Kết quả
dp (skip-connection)	Shortest-path DP trên MỌI cặp khung hình, không chỉ liền kề	Kém hẳn nearest (s16: 0.335 so với 0.656) — cost ở các Δt khác nhau không được hiệu chỉnh cùng thang đo, DP bị “lừa” bởi các đường tắt xa có chi phí thấp giả tạo
ot	Balanced entropic Sinkhorn OT (mass = diện tích tế bào), eps=0.5, tau=None	Đột phá: thắng lap2/lap4 ở MỌI schedule đồng thời, thắng PyUAT tại s32

Chứng minh toán học quan trọng (xác nhận qua kiểm tra độc lập): với một chuỗi hop mà ràng buộc capacity chỉ áp dụng theo từng hop và mục tiêu là tổng chi phí cộng dồn, giải LAP tối ưu độc lập từng hop **đã là tối ưu toàn cục** — không có bộ giải “global flow” nào dùng cùng lượng thông tin per-hop có thể cải thiện thêm. Điều này bác bỏ trước khi cần cài đặt một ý tưởng ban đầu (“global flow decode”) — nó tương đương toán học với lap_decode đã có.

Vì sao OT thắng lap dù cùng dùng per-hop info: OT-balanced ép ràng buộc capacity qua **bảo toàn khối lượng thực sự** (mỗi target phải được giải thích đầy đủ bởi nguồn, trọng số theo diện tích tế bào), không có tham số cap tùy chỉnh phải chọn tay theo từng schedule như LAP.

5. Kết quả cuối cùng

Macro-average trên M3+M4 (embed) / M0–M4 (PyUAT), 3 schedule ends / s16 / s32:

Method	ends	s16	s32
embed_v3_lap2 (baseline learned cost)	0.373	0.706	0.577
embed_v3_ot_e0.5_tinf (chốt)	0.400	0.744	0.668
pyuat_FO+G+O+DD (ILP đa likelihood)	0.313 / 0.151*	0.802	0.589

* PyUAT có hai biến thể tại ends: cấu hình chính (0.313) và biến thể nn (0.151) — cả hai đều thấp hơn hẳn learned embedding, minh chứng cho việc PyUAT collapse khi thiếu quan sát.

Đây là cấu hình **đầu tiên trong toàn bộ nỗ lực nghiên cứu** mà embedding thắng hoặc hòa ở mọi chế độ thừa dữ liệu cùng lúc — trước đó luôn là đánh đổi (embedding tốt ở ends, PyUAT tốt ở s16/s32).

Việc còn dang dở: xác nhận cải tiến OT-decode nhất quán trên embed_v2 (không chỉ v3), và fine-tune eps quanh 0.5 — cả hai job bị hủy do tải máy dùng chung tăng đột biến (load average 83 → 243+, do tiến trình của user khác, không phải lỗi phương pháp). eps=0.5 được chốt làm giá trị an toàn, đã kiểm chứng đầy đủ.

6. Insight chính

1. **Vận tốc là tín hiệu đơn lẻ có đòn bẩy cao nhất** — không phải kiến trúc phức tạp hơn hay huấn luyện lâu hơn.
2. **Compounding error không phải nguyên nhân** của điểm yếu moderate-sparsity — chất lượng cost per-hop mới là gốc rễ.
3. **Giải per-hop capacitated LAP đã tối ưu toàn cục** cho mục tiêu chuỗi cộng dồn — hướng “global solver thông minh hơn” dùng cùng thông tin là ngõ cụt đã được chứng minh, không cần thử nghiệm.
4. **Thông tin xuyên khung hình không liền kề (skip-connection) chỉ có ích nếu cost được hiệu chỉnh cùng thang đo** — DP thất bại vì thiếu điều này, không phải vì ý tưởng sai.
5. **Ràng buộc capacity “mềm” cần cơ chế bảo toàn khối lượng thật (OT), không chỉ softmax** — soft belief propagation thất bại vì thiếu ràng buộc cứng.

6. Một cost học được + OT decode duy nhất có thể thay thế nhu cầu chuyển đổi phương pháp theo độ thừa dữ liệu — đây là đóng góp cốt lõi cho câu chuyện CVPR.

7. Hạn chế

- **Single dataset:** toàn bộ kết quả trên One-in-a-Million (OIAM); chưa kiểm chứng trên bất kỳ dataset Cell Tracking Challenge (CTC) nào khác.
- **Oracle mask/feature:** shape/vị trí lấy từ ground-truth mapping, không phải từ pipeline detector/segmentation thật.
- **s16 vẫn thua PyUAT** ~6 điểm (0.744 vs 0.802).
- **Chưa xác nhận generalize sang embed_v2** và **chưa fine-tune xong eps** — bị chặn bởi tải máy dùng chung, không phải giới hạn phương pháp.
- **Chưa có so sánh với Trackastra** hay khung so sánh hiệu năng tính toán (Pareto: embed+OT rẻ hơn ILP bao nhiêu, đổi lấy bao nhiêu điểm accuracy).

8. Research Question cuối cùng

Một learned per-cell embedding cost duy nhất, decode bằng balanced entropic optimal transport, có tạo ra một phương án **Pareto-dominant** (accuracy đổi lấy chi phí tính toán) so với tracker ILP đa khung hình khớp nối, **trên toàn bộ phổ độ thừa dữ liệu** — và điều này có còn đúng khi kiểm chứng trên nhiều dataset, và dưới điều kiện detection thực tế (không oracle)?

Đây là câu hỏi quyết định liệu hướng này đủ chín để viết bài CVPR Bio-Image: nếu pattern (thắng ends/s32, cạnh tranh s16, rẻ hơn nhiều lần) lặp lại trên dataset khác, đó là một câu chuyện đủ mạnh cho spotlight; nếu không, gap s16 sẽ là vấn đề nhỏ nhất trong danh sách.

9. Next Actions

Ưu tiên	Hành động	Vì sao
Cao	Chạy lại pipeline (embed_v3 + OT decode) trên 1–2 dataset CTC khác	Rào cản lớn nhất cho CVPR — không có nó, số liệu chỉ là single-dataset overfitting risk
Cao	Thay oracle mask bằng detector/segmentation thật	Reviewer Bio-Image sẽ hỏi ngay; hiện tại chỉ là method development
Trung bình	Đo chi phí tính toán (wall-clock/FLOPs) embed+OT vs PyUAT ILP mỗi window	Biến câu chuyện thành Pareto frontier rõ ràng, không chỉ accuracy
Trung bình	Retry xác nhận OT-decode trên embed_v2 + fine-tune eps khi tải máy giảm	Củng cố tính nhất quán của kết quả, chi phí thấp (đã có script)
Thấp	So sánh với Trackastra	Cần thiết cho related-work đầy đủ nhưng không chặn các bước trên
Thấp	Thử thêm feature appearance/CNN để thu hẹp gap s16	Lợi ích biên giảm dần đã thấy rõ (v2 → v3 chỉ +0.014); ưu tiên thấp hơn multi-dataset